

FamilyHub households and invitations

Resumen

- Endpoints implementados:
- POST /households
- POST /households/:household_id/invitations
- POST /invitations/validate
- Alias adicionales:
- POST /api/households
- POST /api/households/:household_id/invitations
- POST /api/invitations/validate
- Auth:
- JWT obligatorio en los 3 endpoints

Archivos modificados

- backend/index.js
- backend/src/controllers/households.controller.js
- backend/src/routes/households.js
- backend/src/routes/invitations.js

Decisiones tecnicas

- Se reutilizo el middleware JWT existente.
- Se mantuvo el cliente actual de Supabase.
- No se agregaron dependencias nuevas.
- tipo se valido con los valores reales hoy usados por el frontend:
- nucleo
- abuelos
- separados
- rol_asignado no tenia referencias claras en el repo, asi que:
- se valida que no venga vacio
- la validacion real final la hace la base si existe enum o check constraint
- Se agrego compensacion simple para evitar estados inconsistentes:
- si falla household_members al crear hogar, se intenta borrar el household
- si falla el marcado de invitacion usada, se intenta borrar el household_member creado

Endpoints implementados

POST /households

- Valida nombre
- Valida tipo
- Crea household
- Crea household_member con rol = coordinador
- Devuelve hogar + member

POST /households/:household_id/invitations

- Verifica existencia del hogar
- Verifica que el usuario autenticado sea coordinador
- Genera token UUID
- Crea invitacion con expiracion a 24 horas
- Devuelve:
- token
- link
- expires_at

POST /invitations/validate

- Busca invitacion por token
- Verifica:
- exista
- no este vencida
- no este usada
- Verifica que el usuario no sea ya miembro
- Crea member con el rol del token
- Marca la invitacion como usada
- Devuelve:
- invitacion
- member
- hogar

Ejemplos de request

Crear hogar

bash

```
curl --request POST "http://localhost:3000/households" \
--header "Authorization: Bearer TU_JWT" \
--header "Content-Type: application/json" \
--data '{"nombre":"Familia Garcia","tipo":"nucleo"}'
```

Crear invitacion

bash

```
curl --request POST "http://localhost:3000/households/HOUSEHOLD_ID/invitations" \
--header "Authorization: Bearer TU_JWT" \
--header "Content-Type: application/json" \
--data '{"rol_asignado":"adulto"}'
```

Validar invitacion

bash

```
curl --request POST "http://localhost:3000/invitations/validate" \
--header "Authorization: Bearer TU_JWT" \
--header "Content-Type: application/json" \
```

```
--data "{\"token\":\"TOKEN_UUID\"}"
```

Test manual

1. Hacer login y copiar access_token.
2. Crear hogar con POST /households.
3. Confirmar 201 y guardar household.id.
4. Crear invitacion con POST /households/:household_id/invitations.
5. Confirmar 201, guardar token.
6. Loguear otro usuario.
7. Validar token con POST /invitations/validate.
8. Confirmar 200 y revisar member creado.
9. Repetir mismo token y confirmar 409.
10. Probar nombre vacio y tipo invalido.
11. Probar usuario no coordinador creando invitacion y confirmar 403.

Riesgos menores

- El schema real no esta versionado en el repo, asi que la validacion de rol_asignado se apoya en la base si existe enum o check constraint.
- El backend usa SUPABASE_KEY de servidor, por eso la seguridad fuerte sigue descansando en los checks manuales del endpoint y en la base.
- No pude ejecutar pruebas reales contra Supabase desde este entorno por bloqueo de red.